

プロジェクト PUSH_SWAPのスケー (/PROJECTS/42NEXT-PUSH_SWAI

このチームの生徒2名を評価する必要があります。



Gitリポジトリ

git@vogosphere-v2.42tokyo.jp:vogosphere/intra-uuid-8b537f10-1

導入

以下のルールを遵守してください。

- 評価プロセス全体を通して、礼儀正しく、丁寧に、敬意を払い、建設的な態度を保ってください。コミュニティの健全な運営は、こうした態度にかかっています。

- 評価対象の学生またはグループのプロジェクトにおける潜在的な問題点を特定してください。特定された問題について、時間をかけて話し合い、議論して；トの指示や機能範囲

について、同僚によって理解の仕方が異なる場合があることを考慮してください。常に先入る限り正直に評価してください。ピア評価を真剣に受け止めてこそ、教育は効果を発揮しま

ガイドライン

- 評価対象の学生またはグループの Git リポジトリに提出された作品のみを採点してください。

- Git リポジトリが学生のものであり、プロジェクトが想定どおりのものであることを確認してください。さらに、「git clone」が空のフォルダで実行されていることを確認してください。

- 悪意のあるエイリアスが使用されておらず、公式リポジトリの内容ではないものを評価させられていないことを注意深く確認してくださ

- 予期せぬ事態を避けるため、また該当する場合は、採点を容易にするために使用されたスクリプト(テスト用または自動化用のスクリプト)と一緒に確認してください。

- 評価する課題を完了していない場合は、評価プロセスを開始する前に、主題全体を読まなければなりません。

- 利用可能なフラグを使用して、空のリポジトリ、機能しないプログラム、ノルムエラー、不正行為などを報告してください。これらの場合、評価プロセスは終了し、最終成績は 0、不正行為の場合は -42 となります。ただし、不正行為を除き、学生は提出された作品と一緒に確認し、今後繰り返されるべきではない間違いを特定することを強く推奨します。

メモリリークがないことも確認する必要があります。ヒープに割り当てられたメモリは、実行終了前に適切に解放されなければなりません。leaks、valgrind、e_fenceなど、コンピュータで利用可能なさまざまなツールを使用してください。メモリリークが検出されグを選択してください。

添付ファイル

📄 主題.pdf (https://cdn.intra.42.fr/pdf/pdf/201834/en.subject.pdf)

📄 フェドラチェッカー (https://cdn.intra.42.fr/document/document/47390/fedora_checker)

 チェッカーマック (https://cdn.intra.42.fr/document/document/47391/checker_Mac)

 チェッカー_linux (https://cdn.intra.42.fr/document/document/47392/checker_linux)

予備

評価を開始する前に、必ず主題全体を読み、理解していることを確認してください。

グループ

評価は、採点対象となる両方の学習者が立ち会う中で行われます。
このように、他者との交流を通して誰もが成長していくのです。

- 弁論の際には、両方の学習者が出席しなければならない。
- 報告なし：0、レビューは終了しました。
- 演習が機能しなくなった時点で、レビューは終了します。以下の演習のコードは閲覧にはなりません。

 はい

 いいえ

検証

グループプロジェクトの検証：

- リポジトリの貢献者として、学習者が正確に2人登録されていることを確認してください
 - 両方の学習者が、実装されたコードのどの部分についても説明し、擁護できることを証明
 - README.mdファイルに各学習者の貢献内容が明確に記載されていることを確認して、
 - 両学習者は、自身の担当部分だけでなく、コードベース全体を理解していることを証明す
- ん。これらの要件のいずれかが満たされない場合、評価は不合格となります。

 はい

 いいえ

README.md 準拠チェック

リポジトリのルートにREADME.mdファイルが存在し、かつ以下の内容がすべて含まれてい

- 最初の行は斜体で、次のように正確にフォーマットされています:このアクティビティ、<login2> によって 42 カリキュラムの一部として作成されました(必要な学習者は正確に
 - 活動の目的を説明し、簡単な概要を提供する「説明」セクション。
 - コンパイル、インストール、および／または実行に関する詳細情報を記載した「手順
 - 「リソース」セクションでは、参考文献（ドキュメント、チュートリアルなど）を一
- うに使用されたか、どのタスクやプロジェクトのどの部分で使用されたかを具体的に記
- このアクティビティで選択されたアルゴリズム（単純なO(n²)、中程度のO(n√n)、複雑適応戦略）の詳細な説明と正当性。
 - 各学習者のプロジェクトへの貢献を明確に記録すること。

 はい

 いいえ

必須項目

注意：ディフェンス中は、セグメンテーション違反、予期せぬクラッシュ、早期終了、制御
られません。これらが発生した場合、最終成績は0点となります。適切なフラグを使用して
はディフェンス全体に適用されます。

ノルミネット

Norminetteを実行してください。エラーが発生した場合は、評価はここで停止します。
コードの実装について議論を続けることはできますが、
課題は採点されません。

 はい

 いいえ

コンピレーション

Makefile が存在し、NAME、all、clean、fclean、re という通常のルールが含まれていること
Makefile

は、-Wall、-Wextra、-Werror フラグを使用してプロジェクトをコンパイルし、再リンクを行
す。

「make」を実行してコンパイルをテストし、push_swap 実行可能ファイルが作成されるこ
い。

「make clean」、「make fclean」、「make re」が正しく動作することを確認してください。
コンパイルエラーが発生した場合、または Makefile が正しく動作しない場合は、評価はここ
コードの実装について議論を続けることはできますが、この
課題は採点されません。

✓ はい

✕ いいえ

メモリリークチェック

弁護活動全体を通して、push_swap が使用するメモリ量（例えば、top コマンドを使用）に注意を払い、異常を検出し、割り当てられたメモリが適切に解放されていることを確認してください。重大なメモリリークが見つかった場合、最終評価は 0 点となります。

注：実装によっては軽微なメモリの問題は許容される場合がありますが、重大なメモリリークやクラッシュが発生した場合は、エラーとして処理されるべきです。

✓ はい

✕ いいえ

エラー管理

このセクションでは、push_swapのエラー管理機能を評価します。これらのテストのほとんどが失敗した場合、このセクションの点数は加算されません。

注：4つのエラーケースのうち、少なくとも3つは正しく処理される必要があります。

- push_swapを数値以外のパラメータで実行してください。プログラムは標準エラー出力に「Error」に続いて「\n」を表示する必要があります。
- 重複した数値パラメータを指定して push_swap を実行してください。プログラムは標準エラー出力に「Error」に続いて「\n」を表示する必要があります。
- push_swapを、MAXINTより大きい値を含む数値パラメータのみを指定して実行してください。プログラムは標準エラー出力に「Error」に続いて「\n」を表示する必要があります。
- パラメータを指定せずにpush_swapを実行してください。プログラムは何も表示せず、プロンプトを返さなければなりません。

✓ はい

✕ いいえ

戦略選択 - 基本テスト

戦略選択フラグをテストしてください。これらのテストの大部分が失敗した場合、このセクションではポイントは付与されません。

- "\$>./push_swap --simple 5 4 3 2 1" を実行して、数値をソートした有効な出力が生成されることを確認してください。
- "\$>./push_swap --medium 5 4 3 2 1" を実行して、数値をソートした有効な出力が生成されることを確認してください。
- "\$>./push_swap --complex 5 4 3 2 1" を実行して、数値をソートした有効な出力が生成されることを確認してください。
- 「\$>./push_swap --adaptive 5 4 3 2 1」を実行して、数値をソートした有効な出力が生成されることを確認してください。
- フラグを指定せずに実行した場合、デフォルトで --adaptive 動作が有効になることを確認してください。

注：このセクションに合格するには、5つのテストのうち少なくとも3つが正しく動作する必要があります。

✓ はい

✕ いいえ

同一性テスト - 既にソート済みの入力

既にソート済みの入力に対して、push_swapの動作をテストしてください。ほとんどのテストが失敗した場合、このセクションの点数は加算されません。

- 「\$>./push_swap 42」を実行してください。プログラムは何も表示しないはずです。
- 「\$>./push_swap 2 3」を実行してください。プログラムは何も表示しないはずです。
- 「\$>./push_swap 0 1 2 3」を実行してください。プログラムは何も表示しないはずです。
- 「\$>./push_swap 0 1 2 3 4 5 6 7 8 9」を実行してください。プログラムは何も表示しないはずです。

入力は既にソートされているため、すべてのテストで出力は発生しません（命令数は0）。

注：4つのテストのうち少なくとも3つは正しく動作する必要があります。

✓ はい

✕ いいえ

小さな入力（3つの数値）

3つの数字でテストしてください。提供されているチェッカーバイナリを使用してください。

イントラプロジェクト push_swap 編集

- "\$>ARG="2 1 0"; ./push_swap \$ARG | ./checker_linux \$ARG" を実行します。
チェッカーが「OK」と表示され、命令数が
妥当であることを確認してください（5以下が許容範囲、3以下が良好）。
- 「0 2 1」や「1 0 2」などの他の3桁の数字の組み合わせでテストし
、チェッカーが妥当な命令数で「OK」を表示することを確認してください。

注：命令数に多少のばらつきがあるのは正常です。まずは正確性を重視してください。

☑ はい ☐ いいえ

中程度の入力値（5つの数値）

5つの数字でテストしてください。提供されているチェッカーバイナリを使用してください。

- "\$>ARG="1 5 2 4 3"; ./push_swap \$ARG | ./checker_linux \$ARG" を実行します。
チェッカーが「OK」と表示し、命令数が
妥当であることを確認してください（15以下が許容範囲、12以下が良好）。
- 5つの乱数の組み合わせを2〜3種類試してください。
チェッカーに「OK」と表示され、命令数が妥当であることを確認してください。
例：「5 1 4 2 3」または「3 5 1 4 2」を試してみてください。

注：アルゴリズムによって命令数が異なる場合があります。
完全な最適化よりも正確性が重要です。

☑ はい ☐ いいえ

ベンチマークモードと障害計算

ベンチマークモードの機能をテストしてください。これは必須要件ではありません
が、課題の趣旨に沿って実装する必要があります。

- "\$>./push_swap --bench --simple 5 4 3 2 1 2>/dev/null" を実行して、
標準出力にソート命令が出力されることを確認してください。
- 「\$>./push_swap --bench --simple 5 4 3 2 1 2>bench.txt >/dev/null && cat bench.txt」：
、ベンチマーク出力に以下の内容のほとんどが含まれていることを確認してください。
 - 障害の割合（存在すべき）
 - 戦略名と複雑性クラス
 - 総操作回数
 - 個々の操作のカウント
- 既知の入力値を用いたテストの障害度計算：
 - ソートされた入力「1 2 3 4 5」の場合、無秩序度は0.00%に近い値であるべきで
 - 逆順にソートされた入力「5 4 3 2 1」の場合、無秩序度は100.00%に近い値にな

注：主要な情報が含まれている場合は、出力形式に若干の差異があっても許容されます。

☑ はい ☐ いいえ

大量の入力（100個の数値）

100個の乱数でテストしてください。提供されているチェッカーバイナリを使用してくださ
以下のコマンドで乱数を生成します：shuf -i 1-500 -n 100

- 異なる乱数セットを用いて、テストを2〜3回実行してください。
- チェッカーがすべてのテストに対して「OK」と表示していることを確認してください
- このテストに合格するには、プログラムは2000回未満の演算で済む必要があります。
- 1500未満なら良好、700未満なら非常に優れたパフォーマンスです。

注：実行結果に多少のばらつきが生じるのは正常です。全体的な正確性を重視してください。

☑ はい ☐ いいえ

コードレビューとアルゴリズムの説明

学習者に、そのアプローチについて簡単に説明してもらいましょう。

- --simple戦略はどのように機能するのでしょうか？（ $O(n^2)$ のアプローチになるはずで
- --medium戦略はどのように機能するのでしょうか？（ $O(n\sqrt{n})$ のアプローチになるはず
- --complex戦略はどのように機能するのでしょうか？（ $O(n \log n)$ のアプローチになるはず
- --adaptive戦略は、どの方法を使用するかをどのように選択するのですか？

学習者は、自身のアルゴリズムについて基本的な説明ができる必要があります。
完璧な理論分析を期待するのではなく、実践的な理解に重点を置いてください。
すべての詳細を完璧に説明できなくても問題ありません。

☑ はい ☐ いいえ

戦略フラグのテスト

同じ入力で異なる戦略フラグをテストする：

- 50個の乱数を生成する：shuf -i 1-200 -n 50
- --simple、--medium、--complex フラグを付けて同じ入力をテストします。
- ほとんどの場合、正しくソートされた有効な出力を生成するはずです。
- 一般的に、--complex は --simple よりも少ない命令数で済みます。
- --adaptive フラグ (またはフラグなし) は機能し、自動的に選択されます

注：すべての戦略が完全に最適化されていなくても問題ありません。
重要なのは、それらが機能し、何らかのパフォーマンスの違いを示すことです。

☒ はい ☐ いいえ

非常に大きな入力値（500個の数値）

500個の乱数でテストしてください。提供されているチェッカーバイナリを使用していただき
生成方法は以下のとおりです：shuf -i 1-1000 -n 500

- 異なる乱数セットを使用してテストを2回実行する
- 両方のテストでチェッカーが「OK」と表示することを確認してください。
- このテストに合格するには、プログラムは12000回未満の演算で済む必要があります。
- 8000未満は良好、5500未満は優れた性能です。

注：これは難易度の高いテストです。パフォーマンスには多少のばらつきが予想されます。
まずは正答率を重視し、次にパフォーマンスを重視してください。

☒ はい ☐ いいえ

ライブコーディングの迅速な修正

レビュー対象者に、スタックをソートするために必要な操作の総数のみを表示し、
実際の操作は表示しない新しいフラグ「--count-only」をpush_swapプログラムに追加するよ
い。
例えば、「./push_swap --count-only 3 2 1」を実行すると、
操作の全リストではなく「3」のみが出力されるはずです。

レビュー対象者は、関連する構文解析および出力コードを特定し
、必要な修正を行い、
いくつかのテストケースを用いて正しく動作することを示す必要があります。
デモンストレーションを含めた作業全体は、
10分以内に完了する必要があります。この手順は遵守され、すべてが正しく動作しましたが

☒ はい ☐ いいえ

ボーナス

注意：ディフェンス中は、セグメンテーション違反、その他予期せぬ、時期尚早な、制御不
プログラムの終了は発生しません。発生した場合は最終評価は0となります。適切なフラグを
このルールはディフェンス全体を通して有効です。必須部分が「優秀」である場合に限り、
す。これは、必須部分を最初から最後まで完了する必要があり、たとえ誤った使用や不適切
ラー管理が完璧でなければならないことを意味します。このディフェンス中に必須部分が満
合、ボーナスセクションは完全に無視されます。

チェッカープログラム - エラー管理

このセクションでは、チェッカーのエラー処理能力を評価します。エラー処理能力が
1つでも不十分な場合は、この
セクションの点数は加算されません。次のセクションに進んでください。

- 数値以外のパラメータを指定してチェッカーを実行してください。プログラムは
標準エラー出力に「Error」の後に改行文字「\n」を表示する必要があります。
- 重複した数値パラメータを指定してチェッカーを実行してください。プログラムは
標準エラー出力に「Error」に続いて「\n」を表示する必要があります。
- MAXINTより大きい値を含む数値パラメータのみを指定してチェッカーを実行してくた
。プログラムは
標準エラー出力に「Error」に続いて「\n」を表示する必要があります。
- パラメータを指定せずにチェッカーを実行してください。プログラムは
何も表示せず、プロンプトを返さなければなりません。
- 有効なパラメータを指定してチェッカーを実行し、命令フェーズ中に存在しないアクシ
ョンを実行してください。
。プログラムは
標準エラー出力に「Error」に続いて「\n」を表示する必要があります。

- 有効なパラメータを指定してチェッカーを実行し、命令フェーズ中に、アクションの前後に1つまたは複数のスペースを記述してください。プログラムは、標準エラー出力に「Error」に続いて「\n」を表示する必要があります。

✓ はい

✕ いいえ

チェッカープログラム - 誤ったテスト

このセクションでは、リストをソートしない命令リストを処理するチェッカーの能力を評価以下の2つのテストを実行してください。少なくとも1つが失敗した場合、このセクションのポイントは付与されません。次のセクションに進んでください。

指示表示中は、読み上げを停止するためにCtrl+Dを押すことを忘れないでください。

- 次のコマンド「\$>./checker 0 9 1 8 2 7 3 6 4 5」でチェッカーを実行し、次の有効なアクションリスト「[sa, pb, rrr]」を入力してください。チェッカーには「KO」と表示されるはずですが、
- 有効なリストを任意のパラメータとしてチェッカーを実行し、次に整数を順序付けしない有効な命令リストを作成します。チェッカーは「KO」と表示するはムが、このスケールに含まれるテストでのみ正解するように開発されていないことを必要があります。検証する前に、いくつかの順列でこのテストを数回繰り返してください。

✓ はい

✕ いいえ

チェッカープログラム - 正しいテスト

このセクションでは、リストをソートする一連の命令を管理するチェッカーの能力を評価し2つのテストを実行してください。少なくとも1つが失敗した場合、このセクションのポイントは付与されません。次のセクションに進んでください。

指示表示中は、読み上げを停止するためにCtrl+Dを押すことを忘れないでください。

- 次のコマンド「\$>./checker 0 1 2」でチェッカーを実行し、何も入力せずにCtrl+Dを押してください。プログラムに「OK」と表示されるはずですが、
- 次のコマンド「\$>./checker 0 9 1 8 2」でチェッカーを実行し、次の有効なアクションリスト「[pb, ra, pb, ra, sa, ra, pa, pa]」を書き込みます。プログラムは「OK」と表示するはずですが、
- 有効なリストを任意のパラメータとしてチェッカーを実行し、整数を正しく並べ替える有効な命令リストを作成してください。チェッカーは「OK」ます。このプログラムが、この評価尺度に含まれるテストでのみ正解するように開発具体的に確認する必要があります。検証する前に、いくつかの順列でこのテストを数回繰り返してください。

✓ はい

✕ いいえ

評価

防御に対応する旗を確認することを忘れないでください

✓ わかりました

★素晴らしいプロジェクト

空っぽの仕事

📌 未完了の作業

🌀 無効なコンパイル

📄 ノルム

📁 雰囲気

⬆️ 懸念すべき状況

💧 リーク

🚫 禁止されている機能

💬 コードのサポート

結論

この評価についてコメントを残してください（最大2048文字）

自分の実装するときのた
めのいい知識が付きまし

評価を完了する

API一般利用規約
(<https://profile.intra.42.fr/legal/terms/33>)

クッキーの使用に関する宣言
(<https://profile.intra.42.fr/legal/terms/2>)

プライバシーポリシー
(<https://profile.intra.42.fr/legal/terms/5>)

サイト利用規約
(<https://profile.intra.42.fr/legal/terms/6>)

手
(<https://profile.ir>)